S.K.A.K. Madhushan

E/21/245

CO226 – DATABASE SYSTEMS LAB 06

Assume that you're working for a university database system that stores information about students, courses, and enrollments.

Table Name	Description
students	Stores student information
courses	Stores course details
enrollments	Associates students with courses
instructors	Stores instructor details
course_instructors	Maps instructors to courses

```
mysql> select * from students;
+------------+--------+------------+----------------------+------+
| student_id | name   | dob        | email                | gpa  |
+------------+--------+------------+----------------------+------+
|         49 | Lahiru | 2002-09-01 | e21049@eng.pdn.ac.lk | NULL |
|        138 | FIKRY  | 2002-01-15 | e21138@eng.pdn.ac.lk | 3.95 |
|        217 | Ishan  | 2002-06-21 | e21217@eng.pdn.ac.lk | 3.35 |
+------------+--------+------------+----------------------+------+
3 rows in set (0.027 sec)

mysql> select * from courses;
+-----------+----------------------+---------+
| course_id | course_name          | credits |
+-----------+----------------------+---------+
|       224 | Computer Architecture |       4 |
|       225 | Software Construction |       3 |
|       226 | Database Systems      |       3 |
+-----------+----------------------+---------+
3 rows in set (0.007 sec)

mysql> select * from enrollments;
+------------+-----------+-------------+-------+
| student_id | course_id | enroll_date | grade |
+------------+-----------+-------------+-------+
|         49 |       226 | 2024-02-15  | NULL  |
|        138 |       224 | 2024-02-01  | A     |
|        217 |       225 | 2024-02-01  | B     |
+------------+-----------+-------------+-------+
3 rows in set (0.008 sec)

mysql> select * from instructors;
+---------------+----------------+-----------------------+
| instructor_id | name           | department            |
+---------------+----------------+-----------------------+
|            10 | Saadia Jameel  | Computer Engineering  |
|            11 | Shakthi Perera | Electrical Engineering |
+---------------+----------------+-----------------------+
2 rows in set (0.009 sec)
```

```
mysql> select * from course_instructors;
+-----------+---------------+
| course_id | instructor_id |
+-----------+---------------+
|       224 |            10 |
|       225 |            11 |
+-----------+---------------+
2 rows in set (0.012 sec)
```

1.

a. List all students details.
b. Display all courses that offer more than 3 credits.
c. Show all enrollments with student name, course name, and grade.

1. a.

```
mysql> select * from students;
+------------+--------+------------+----------------------+------+
| student_id | name   | dob        | email                | gpa  |
+------------+--------+------------+----------------------+------+
|         49 | Lahiru | 2002-09-01 | e21049@eng.pdn.ac.lk | NULL |
|        138 | FIKRY  | 2002-01-15 | e21138@eng.pdn.ac.lk | 3.95 |
|        217 | Ishan  | 2002-06-21 | e21217@eng.pdn.ac.lk | 3.35 |
+------------+--------+------------+----------------------+------+
3 rows in set (0.027 sec)
```

b.

```
mysql> select * from courses where credits>3;
+-----------+----------------------+---------+
| course_id | course_name          | credits |
+-----------+----------------------+---------+
|       224 | Computer Architecture |       4 |
+-----------+----------------------+---------+
1 row in set (0.044 sec)
```

c.

```
mysql>  select s.name as student_name, c.course_name, e.grade
    ->  from enrollments e
    -> join students s on e.student_id = s.student_id join courses c on e.course_id = c.course_id;
+--------------+----------------------+-------+
| student_name | course_name          | grade |
+--------------+----------------------+-------+
| Lahiru       | Database Systems     | NULL  |
| FIKRY        | Computer Architecture | A     |
| Ishan        | Software Construction | B     |
+--------------+----------------------+-------+
3 rows in set (0.042 sec)
```

2.

a. Show each course with the number of enrolled students.
b. Display average GPA of students per course.

a.

```
mysql> SELECT c.course_name, COUNT(e.student_id) AS num_students
    -> FROM courses c
    -> LEFT JOIN enrollments e ON c.course_id = e.course_id
    -> GROUP BY c.course_name;
+-----------------------+--------------+
| course_name           | num_students |
+-----------------------+--------------+
| Computer Architecture |            1 |
| Software Construction |            1 |
| Database Systems      |            1 |
+-----------------------+--------------+
3 rows in set (0.085 sec)
```

We are selecting the course name and counting how many students are in each course, We match each course with enrollments using the course ID. Even if no one enrolled, the course still shows up because it's a **LEFT JOIN.**We group the results course-by-course.

3.

a. Find students who enrolled after '2024-02-10'.
b. Retrieve names of students who got the highest GPA.

a.

```
mysql> SELECT DISTINCT s.*
    -> FROM enrollments e
    -> JOIN students s ON e.student_id = s.student_id
    -> WHERE e.enroll_date > '2024-02-10';
+------------+--------+------------+----------------------+------+
| student_id | name   | dob        | email                | gpa  |
+------------+--------+------------+----------------------+------+
|         49 | Lahiru | 2002-09-01 | e21049@eng.pdn.ac.lk | NULL |
+------------+--------+------------+----------------------+------+
1 row in set (0.037 sec)
```

In here we are selecting all student details but only unique ones. Start from enrollments to find when students joined and Match enrollment records to student info only pick students who enrolled after Feb 10.

b.

```
mysql> SELECT name
    -> FROM students
    -> WHERE gpa = (SELECT MAX(gpa) FROM students);
+-------+
| name  |
+-------+
| FIKRY |
+-------+
1 row in set (0.058 sec)
```

We just want the name of the highest gpa person and for that here we are create a sub query to find it , we use MAX() to find highest GPA from the all students and next we get that highest GPA and find the person who has that .

4.

a. Update Lahiru's GPA to 3.2.
b. Delete enrollments where the grade is NULL

a.

```
mysql> UPDATE students
    -> SET gpa = 3.2
    -> WHERE name = 'Lahiru';
Query OK, 1 row affected (0.112 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from students where name = 'Lahiru';
+------------+--------+------------+----------------------+------+
| student_id | name   | dob        | email                | gpa  |
+------------+--------+------------+----------------------+------+
|         49 | Lahiru | 2002-09-01 | e21049@eng.pdn.ac.lk | 3.20 |
+------------+--------+------------+----------------------+------+
1 row in set (0.006 sec)
```

in here we are updating the Lahiru' s gpa with the value 3.2 which was before NULL and here we just simply use the UPDATE , SET commands to do that

b.

```
mysql> DELETE FROM enrollments
    -> WHERE grade IS NULL;
Query OK, 1 row affected (0.063 sec)
```

```
mysql> select * from enrollments;
+------------+-----------+-------------+-------+
| student_id | course_id | enroll_date | grade |
+------------+-----------+-------------+-------+
|        138 |       224 | 2024-02-01  | A     |
|        217 |       225 | 2024-02-01  | B     |
+------------+-----------+-------------+-------+
2 rows in set (0.006 sec)
```

simply used DELETE keyword to delete the person who has NULL grade and you can see then the row has deleted .

5.

a. Create a view named *student_course_view* showing student name, course name, and grade.
b. Create a stored procedure *enroll_student* to insert a record into enrollments with student_id, course_id, enroll_date, grade.

a.

```
mysql> CREATE VIEW student_course_view AS
    -> SELECT s.name AS student_name, c.course_name, e.grade
    -> FROM enrollments e
    -> JOIN students s ON e.student_id = s.student_id
    -> JOIN courses c ON e.course_id = c.course_id;
Query OK, 0 rows affected (0.155 sec)

mysql> SELECT * FROM student_course_view;
+--------------+-----------------------+-------+
| student_name | course_name           | grade |
+--------------+-----------------------+-------+
| FIKRY        | Computer Architecture | A     |
| Ishan        | Software Construction | B     |
+--------------+-----------------------+-------+
2 rows in set (0.046 sec)
```

We want to make a VIEW which is a like new virtual table without store data (in here it is called as student_course_view) , for that we are start with enrollments (where all the connections are in) and we Connect to the students table for names and Connect to the courses table for course names.

b.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE enroll_student (
    ->     IN p_student_id INT,
    ->     IN p_course_id INT,
    ->     IN p_enroll_date DATE,
    ->     IN p_grade CHAR(1)
    -> )
    -> BEGIN
    ->     INSERT INTO enrollments (student_id, course_id, enroll_date, grade)
    ->     VALUES (p_student_id, p_course_id, p_enroll_date, p_grade);
    -> END;
    -> //
Query OK, 0 rows affected (0.307 sec)

mysql> DELIMITER ;
mysql> CALL enroll_student(138, 226, '2024-03-01', 'A');
Query OK, 1 row affected (0.069 sec)

mysql> SELECT * FROM enrollments
    -> WHERE student_id = 138 AND course_id = 226;
+------------+-----------+-------------+-------+
| student_id | course_id | enroll_date | grade |
+------------+-----------+-------------+-------+
|        138 |       226 | 2024-03-01  | A     |
+------------+-----------+-------------+-------+
1 row in set (0.011 sec)

mysql>
```

In this task we are making a reusable command (procedure) called enroll_student with input parameters.These are the inputs we'll give the procedure (student ID, course ID, date, grade).Then we give what to do when this procedure run ,in BEGIN and END block.take the inputs and stick them into the enrollments table.